

Lambda Expressions

↳ java.util.function

4 Core Interfaces

↳ Function
↳ Consumer
↳ Supplier
↳ Predicate

Function

↳ (Take i/p → o/p)
 $T \rightarrow R$

$(x) \rightarrow x * x$

```
interface Function<T,R> {  
    R apply (T t);  
}
```

Consumer<T>

$T \rightarrow \text{void}$

$(x) \rightarrow \text{System.out.println}(x);$

```
public interface Consumer<T> {  
    void apply (T t);  
}
```

Supplier<T>

$\text{void} \rightarrow T$

$() \rightarrow \text{Math.random}();$

```
public interface Supplier<T> {  
    T get();  
}
```

Predicate <T>

T → boolean

$(x) \rightarrow (x \% 2 == 0)$

```
public interface Predicate<T> {  
    boolean test(T t);  
}
```

T → R	Function	Transform
T → void	Consumer	Consume
void → T	Supplier	Supply
T → boolean	Predicate	test

Primitive Functional Interfaces

Function <Integer, Integer> sarak =
 $(x \rightarrow x * x);$

→ Unboxing
→ Operation
→ Autoboxing

→

int
long
double

Function (Primitive Family)
(T → R)

IntFunction <T>

(int → R)

LongFunction <T>

(long → R)

DoubleFunction <T>

(double → R)

ToIntFunction <T>

ToLongFunction <T>

ToDoubleFunction <T>

↙ ↘

T → int
long
double

Primitive Consumer Family

↳ $T \rightarrow \text{void}$

IntConsumer $\rightarrow (int \rightarrow \text{void})$
LongConsumer $(long \rightarrow \text{void})$
DoubleConsumer $(double \rightarrow \text{void})$

Obj IntConsumer
 $(T, int) \rightarrow \text{void}$
Obj LongConsumer
 $(T, long) \rightarrow \text{void}$

Primitive Supplier

↓
↳ $\text{void} \rightarrow T$

IntSupplier $(\text{void} \rightarrow int)$
LongSupplier $(\text{void} \rightarrow long)$
DoubleSupplier $(\text{void} \rightarrow double)$

Primitive Predicate

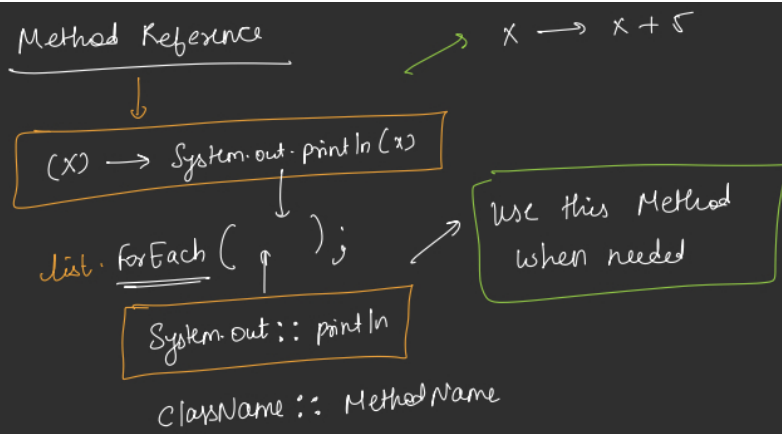
↓
↳ $(T \rightarrow \text{boolean})$
IntPredicate $(int \rightarrow \text{boolean})$
LongPredicate $(long \rightarrow \text{boolean})$
DoublePredicate $(double \rightarrow \text{boolean})$

Primitive Operator Family

↓
IntUnaryOperator $(int \rightarrow int)$
LongUnaryOperator $(long \rightarrow long)$
DoubleUnaryOperator $(double \rightarrow double)$

IntBinaryOperator $((int, int) \rightarrow int)$
— $(\rightarrow , -$
— $- , \rightarrow$

Method Reference



Types of Method Reference

Static Method Reference

$x \rightarrow \text{Math.abs}(x);$

$\Downarrow$

$(\text{Math}::\text{abs})$

Instance Method Reference

$(x) \rightarrow \text{Syso}(x);$

$\text{System.out}::\text{println}$

$\text{str} \rightarrow \text{str.isEmpty}()$

$\text{Predicate}\langle\text{String}\rangle p = \text{String}::\text{isEmpty}()$ ✓

$p.\text{test}(\text{"Hello"});$ (✓)
 $\hookrightarrow \text{false}$

Constructor Reference

$\text{Supplier}\langle T \rangle s = () \rightarrow \text{new ArrayList}\langle\text{Integer}\rangle();$
($\text{void} \rightarrow T$)

$s = \text{ArrayList}::\text{new}$

Functional Composition

int result = $((x+2) * 3)$

add 2
Multiply 3.

Mathematically,

$$\begin{aligned} f(x) &= x+2 \\ g(x) &= x*3 \end{aligned}$$

$$g(f(x))$$

Function $\langle T, R \rangle \longrightarrow R \text{ apply } (T \ t);$

$T \rightarrow R$
(I/P) (O/P)

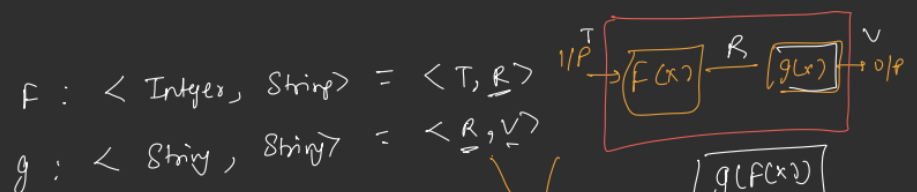
andThen()
Compose()

andThen()

Function $\langle T, R \rangle \quad f = x \rightarrow x+2$
Function $\langle R, V \rangle \quad g = x \rightarrow x*2$

$$f.\text{andThen}(g) = g(f(x))$$

First F, and then g $\hookrightarrow g.\text{apply}(f.\text{apply}(2))$



$f.\text{andThen}(g)$

$$g(f(x))$$

✓✓

$g.\text{andThen}(f)$

$$f(g(x))$$

✗

Function $\langle T, V \rangle$

Compose C)

$$\text{Function } f = x \rightarrow x + 2$$

$$g = x \rightarrow x * 2$$

$$f.\text{compose}(g) \rightarrow \boxed{f(g(x))}$$

$$f.\text{andThen}(g) \rightarrow \boxed{g(f(x))}$$

$$\rightarrow [(x * 2) + 2]$$

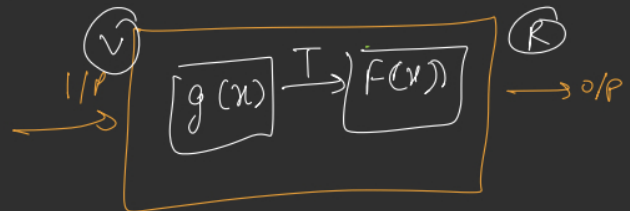
$$\rightarrow [[x + 2] * 2]$$

$$\text{Function } \langle T, R \rangle \quad f$$

$$\text{Function } \langle V, T \rangle \quad g$$

$$f.\text{compose}(g) \rightarrow \underline{f(g(x))}$$

AndThen()
compose()



↳ method pipeline -

$$f1.\text{andThen}(f2).\text{andThen}(f3).\text{apply}(e)$$

andThen()
compose()

↳ Method Reference!

Function < String, String> toUpperCase = String::toUpperCase
Function < String, Integer> length = String::length

\ /

Function < String, Integer> result

↳ toUpperCase.andThen(length)